

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

COMPLEXITY II

WE HAVE BEEN USING Θ ON A PROMISE

SETTLED

We can build the execution table, collapse it to $T(n)$, and read the growth rate off the most-executed line.

OWED

What *is* $\Theta(n^2)$? We have been writing a claim we cannot yet prove- or disprove.

BIG- Θ , FORMALLY

A RECIPE CANNOT SETTLE AN ARGUMENT

YOU CLAIM, I DISAGREE

“Drop the constants” is a habit. If two people apply it and disagree, there is nothing to appeal to.

SOME FUNCTIONS ARE NOT COMPARABLE

Is $5n$ a member of $\Theta(n^2)$? The recipe has no opinion. The definition says no, and says why.

Θ IS A SET, NOT AN OPERATION

$\Theta(g(n))$ is the set of all functions that grow like g . “ $T(n) \in \Theta(n)$ ” is a membership claim.

PROOF OBLIGATIONS ARE EXAM MATERIAL

You will be asked to produce c_1, c_2 and n_0 - or to derive a contradiction from assuming they exist.

The definition is the referee. Everything else today is an application of it.

BIG-Θ

DEFINITION

Let $g(n)$ be a function. We write $\Theta(g(n))$ to denote the set of all functions $f(n)$ for which there exist positive real numbers c_1 , c_2 and an $n_0 \in \mathbb{N}^+$ such that for all natural numbers $n \geq n_0$,

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

C_1 : THE FLOOR

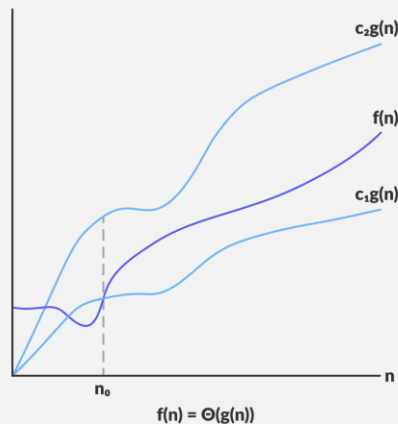
f is eventually at least this multiple of g .
This is the lower bound.

C_2 : THE CEILING

f is eventually at most this multiple of g .
This is the upper bound.

N_0 : EVENTUALLY

Small n is exempt. The claim is about large inputs only.



WHAT THE QUANTIFIERS ARE DOING

CLAUSE	KIND	WHAT IT OBLIGES YOU TO DO
there exist c_1, c_2, n_0	Existential	Any working triple proves the claim.
for all $n \geq n_0$	Universal	Every large n , no exceptions. One counterexample past n_0 kills the claim.
$0 \leq c_1 g(n)$	Sign condition	Rules out functions that go negative. Running times never do.

The asymmetry is the whole game: to prove membership you produce numbers; to disprove it you must show no numbers could ever work. That second job is what proof by contradiction is for.

PROVE YOUR FUNCTION BELONGS

THE SETTING

I am the gatekeeper of the set $\Theta(n^2)$. You arrive with your favorite function. To get in, you satisfy the definition.

YOUR FUNCTION

$$f(n) = 3n^2 + 4n$$

Two bounds to produce, and you may pick the constants.

Find c_1, c_2, n_0 with $c_1n^2 \leq 3n^2 + 4n \leq c_2n^2$ for all $n \geq n_0$

Recall: Find c_1, c_2, n_0 with $c_1n^2 \leq 3n^2 + 4n \leq c_2n^2$ for all $n \geq n_0$

THE LOWER BOUND IS ALMOST FREE

$$3n^2 + \cancel{4n} \geq 3n^2$$

for all $n \geq 0$

so take $c_1 = 3$ and $n \geq 0$

WHY IT WORKS

$4n$ is non-negative for every $n \geq 0$. Throwing away a non-negative term can only make the expression smaller- which is exactly the direction a lower bound needs.

Rule of thumb. For a lower bound, discard positive terms. For an upper bound, discard negative ones.

Recall: Find c_1, c_2, n_0 with $c_1 n^2 \leq 3n^2 + 4n \leq c_2 n^2$ for all $n \geq n_0$

THE UPPER BOUND TAKES ALGEBRA

$$\begin{array}{rcl} 3n^2 + 4n & \leq & c_2 n^2 \\ \downarrow & & \downarrow \\ 3n + 4 & \leq & c_2 n \\ \hline (c_2 - 3)n & \geq & 4 \end{array}$$

divide by $n > 0$
gather like terms

CHOOSE, THEN SOLVE

Pick $c_2 = 4$. Then $(c_2 - 3)n = n$, and $n \geq 4$ whenever $n \geq 4$.

Any $c_2 > 3$ would have worked. 4 was chosen because the arithmetic is short.

$$\underline{3n^2} \leq \underline{3n^2 + 4n} \leq \underline{4n^2} \quad \text{for all } n \geq 4 \quad \therefore f(n) = \Theta(n^2)$$

$$f(n) = 3n^3 - 50n + 42$$

LOWER BOUND

$$3n^3 - 50n + 42 \geq 3n^3 - 50n$$

$$(3 - c_1)n^2 \geq 50$$

$$c_1 = 1 \Rightarrow 2n^2 \geq 50 \Rightarrow n \geq 5$$

c_1 must stay below 3, or the left side goes negative and can never clear 50.

UPPER BOUND

$$3n^3 - 50n + 42 \leq 3n^3 + 42$$

$$(c_2 - 3)n^3 \geq 42$$

$$c_2 = 45 \Rightarrow 42n^3 \geq 42 \Rightarrow n \geq 1$$

$-50n$ is non-positive, so dropping it can only make the expression larger.

$$n^3 \leq f(n) \leq 45n^3 \quad \text{for all } n \geq 5 \quad \therefore f(n) = \theta(n^3)$$

FOUR STEPS, EVERY TIME

01 Write the two inequalities

$c_1g(n) \leq f(n)$ and $f(n) \leq c_2g(n)$. Two separate proofs; do not mix them.

02 Simplify by discarding

Positive terms go for the lower bound, negative terms for the upper.

03 Choose the constant, solve for n

Pick a convenient c , gather like terms, read off the threshold.

04 Take $n_0 = \max$ of thresholds

Then state the sandwich.

Step 4 is the one people skip. Without it you have two facts and no conclusion.

WHAT IF ONLY ONE SIDE IS TRUE?

Θ demands both bounds.

Sometimes we only have one- or only want one.

BIG-O, BIG- Ω , AND BOTTLENECKS

One-sided bounds, the sum rule, and the first proof by contradiction of the quarter.

IS $5n$ A MEMBER OF $\Theta(n^2)$?

THE UPPER BOUND HOLDS

$$5n \leq 5n^2 \quad \text{for all } n \geq 1$$

So $c_2 = 5, n_0 = 1$. Nothing wrong here.

THE LOWER BOUND CANNOT

$$\underbrace{5n}_{\text{circled}} \geq c \cdot n^2 \quad \text{for large } n ?$$

No positive c makes this true forever. Proof on the next slide.

$5n$ is bounded above by n^2 and not bounded below by it. Θ has no way to say that. We need a notation for half a sandwich.

$5n$ IS NOT $\Theta(n^2)$ BY CONTRADICTION

THE ARGUMENT

Suppose such a c exists.

$$\begin{array}{ll} 5n \geq c \cdot n^2 & \text{for all } n \geq n_0 \\ \textcircled{5} \geq c \cdot n & \text{divide by } n > 0 \end{array}$$

But $c \cdot n \rightarrow \infty$. **Contradiction.**

THE SHAPE OF THE MOVE

Assume the constants exist. Push the assumption until it asserts that a fixed number dominates an unbounded one. No c survives that.

You cannot disprove membership by trying constants and failing. You disprove it by showing every constant fails at once.

BIG-O AND BIG-Ω

DEFINITION — BIG-O

There exists a positive real c and a positive natural n_0 such that for all $n \geq n_0$,

$$0 \leq f(n) \leq c \cdot g(n)$$

Upper bound only. “Grows no faster than.”

DEFINITION — BIG-Ω

There exists a positive real c and a positive natural n_0 such that for all $n \geq n_0$,

$$0 \leq c \cdot g(n) \leq f(n)$$

Lower bound only. “Grows at least as fast as.”

CLAIM	BOUNDS	WHAT FOLLOWS
$f(n) = \Theta(g(n))$	both	$\Rightarrow f = O(g)$ and $f = \Omega(g)$
$f(n) = O(g(n))$	upper only	$\Rightarrow$ nothing about Θ . $5n = O(n^2)$, but $5n \neq \Theta(n^2)$.
$f(n) = \Omega(g(n))$	lower only	$3n^2 + 4n = \Omega(n^2)$, and also $\Omega(n)$.

TRUE, AND USELESS

TECHNICALLY CORRECT

“Binary search runs in $O(n!)$ time.”

Nothing in the definition is violated. The bound is simply enormous, so it excludes nothing.

WHAT IS BEING ASKED FOR

A **tight** bound. If you can prove Θ , say Θ . Reach for O only when the lower bound is genuinely unknown or genuinely irrelevant.

In practice O is used conversationally where Θ is meant. In this course, when the question says “tightest,” it is testing whether you can produce both bounds.

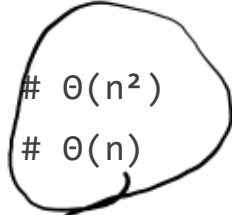
ADDING TWO PIECES: THE MAX RULE

THEOREM

If $f_1(n) = \Theta(g_1(n))$ and $f_2(n) = \Theta(g_2(n))$, then $f_1(n) + f_2(n) = \Theta(g(n))$ where $g(n) = \max\{g_1(n), g_2(n)\}$.

SEQUENTIAL CODE

```
def baz(n):  
    foo(n)    #  $\Theta(n^2)$   
    bar(n)    #  $\Theta(n)$ 
```



THE BOTTLENECK

$$T_{\text{baz}}(n) = \Theta(\max\{n^2, n\}) = \Theta(n^2).$$

foo dominates. Remove it and baz becomes linear immediately- which is exactly how you decide what to optimize.

The same statement holds for O and for Ω . Recall Lecture 2's `triangle_sum`: two loop nests in series, so take the **max**, not the sum.

SAY WHICH, AND WHY

STATEMENT	VERDICT	REASON
$n^2 + n = O(n^3)$	True	Upper bound holds; it is just not tight.
$n^2 + n = \Theta(n^3)$	False	No c_1 works: $n^2 + n \geq c_1 n^3$ fails for large n .
$100n = \Omega(n)$	True	$c = 100, n_0 = 1$. Constants are free (and again, useless).
$2^n = O(3^n)$	True	$2^n \leq 3^n$ for all $n \geq 1$, so $c = 1$.
$3^n = O(2^n)$	False	$3^n/2^n = (3/2)^n \rightarrow \infty$, so no constant c bounds it.

BRUTE FORCE, AND ITS PRICE

THE BROKEN ASSUMPTION

```
def linear_search(arr, target):  
    """Return the index of target in arr, or None if absent."""  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return None
```

Last lecture's procedure says: count the most-executed line. How many times does the loop body run?

IT DEPENDS

target at index 0, it will run 1 time.
If target is absent, it will run n iterations.
Same n, different runtime.

$[1, 3, 7, 15]$

$O(1)$

$O(n)$

$T(n) \Rightarrow f(n)$

We need to ask ourselves three questions

01 Best case

The fastest input of size n . $\Theta(1)$

02 Worse case

The slowest input of size n . $\Theta(n)$

03 Average case

The expected runtime over a stated distribution of inputs.



AVERAGE CASE REQUIRES AN ASSUMPTION

There is a probability $|E|$

THE SITUATION

Assume target is present and equally likely at any of the n positions.

Then, the expected comparisons we will do are:

$$(1 + 2 + \dots + n)/n = (n+1)/2 \in \Theta(n)$$

Change the assumption and you change the answer. Average case is meaningless without a stated distribution - always state it.

WHY WORST CASE IS THE DEFAULT

It's a guarantee. Nothing is slower than this.

It doesn't require knowing your input distribution.

Best case is usually a trivial, useless input.

Adversarial inputs are real: the worst case is what an attacker sends.

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
Mergesort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
Timsort	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$
Heapsort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(1)$
Bubble Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Tree Sort	$\Omega(n \log(n))$	$\Theta(n \log(n))$	$O(n^2)$	$O(n)$
Shell Sort	$\Omega(n \log(n))$	$\Theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
Bucket Sort	$\Omega(n+k)$	$\Theta(n+k)$	$O(n^2)$	$O(n)$
Radix Sort	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$	$O(n+k)$
Counting Sort	$\Omega(n+k)$	$\Theta(n+k)$	$O(n+k)$	$O(k)$
Cubesort	$\Omega(n)$	$\Theta(n \log(n))$	$O(n \log(n))$	$O(n)$

THE MISCONCEPTION

“ O is worst case, Ω is best case, Θ is average case.”

TWO INDEPENDENT AXES

worst case $O(n) \neq$
worst case $\Theta(n)$

{

WHICH INPUT?

best / worst / average

WHICH BOUND?

O upper / Ω lower / Θ tight

READ THE QUESTION CAREFULLY

“WHAT IS THE TIME COMPLEXITY OF X?”

Worst case unless stated.

“GIVE AN O BOUND”

An upper bound; a loose one is true but weak.

“GIVE A TIGHT BOUND”

Θ , and you must justify both sides.

YOUR TURN

```
def has_duplicate(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if arr[i] == arr[j]:  
                return True  
  
    return False
```

Give the best-case and worst-case time complexity in terms of n . Justify each with a specific input.

SOLUTION

```
def has_duplicate(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if arr[i] == arr[j]:  
                return True  
    return False
```

Best case. $\Theta(1)$

$\text{arr}[0] == \text{arr}[1] \rightarrow$ returns on the first inner iteration.

Worst case. $\Theta(n^2)$

no duplicates $\rightarrow$ both loops run out.

$[1, 2, 3] \quad \Theta(n^2)$

$[1, 1, 3, \dots]$